gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843 Regression Testing

Dr. Francisco Gomes de Oliveira Neto (he/him)

Prof. Mary Jean Harrold

- Professor at the Georgia Institute of Technology.
- Foundational studies and research on Regression Testing
- 2007 best software engineering researcher in the world!
- Fierce advocate for women and minorities in computing fields.
- More details in her memorial page.

What we will learn today?

- What is **regression testing**? **Why** is regression testing important?
- How can we make regression testing **more cost-effective**?
- What are the **differences between** test selection, prioritisation and minimisation?
- What are the different **techniques to minimize** test suites?

© Francisco Oliveira Neto - 2018

Part 1:

Cost-effective

Regression Testing

© Francisco Oliveira Neto - 2022

What are Regression Faults?

- Software **functionalities that work might stop working due to the modifications** caused in the software.

- **Regression faults** are faults caused because of a modification.
- Can be a direct or indirect (**side-effect**) consequence of the modification.

© Francisco Oliveira Neto - 2018

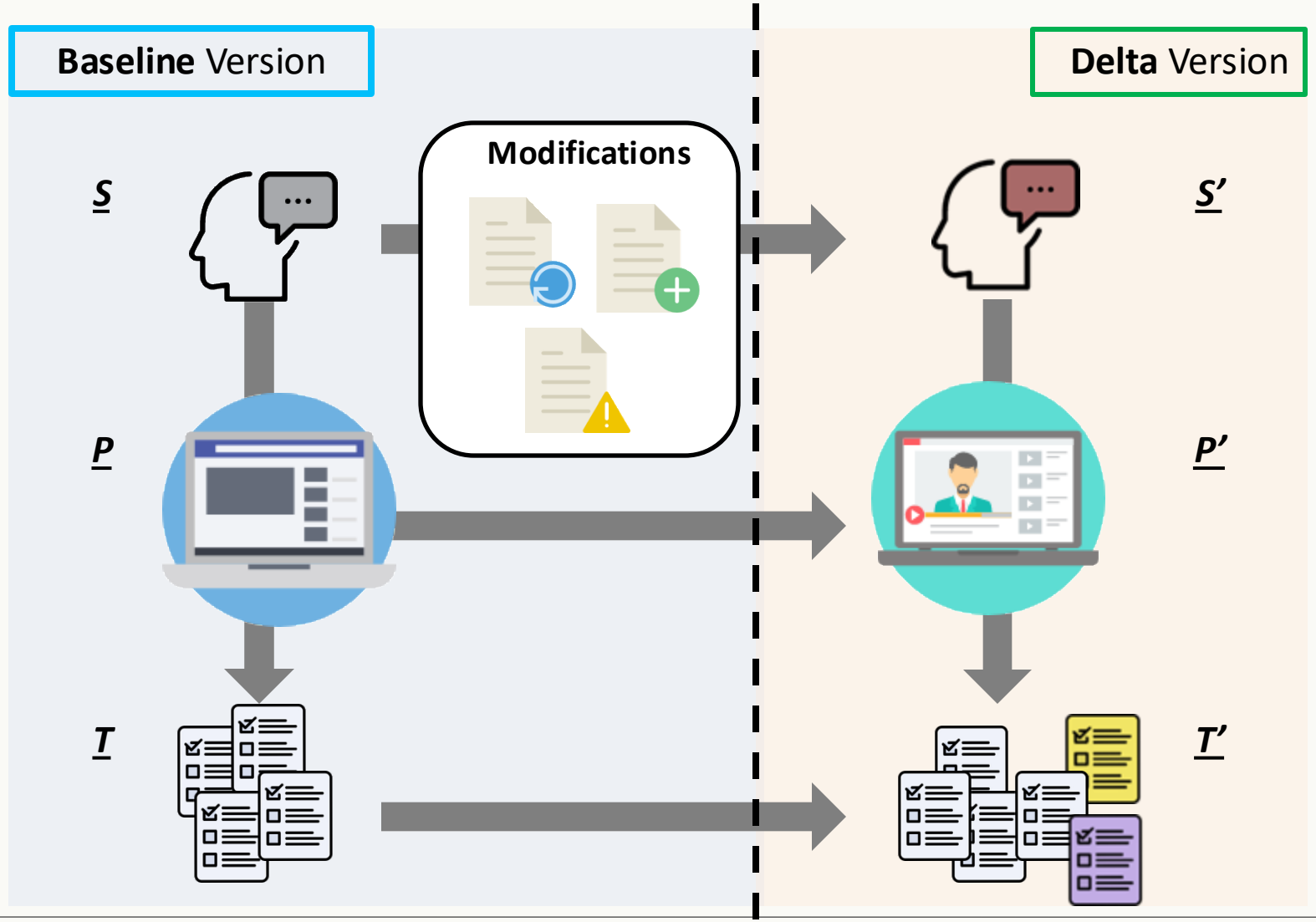
What is Regression Testing?

- **Regression testing** is the activity focused on identifying regression faults.

- Regression testing can be done **across all levels** of testing. It is mainly done at unit and integration level, in **combination with Continuous Integration (CI)**.

© Francisco Oliveira Neto - 2022

Important Concepts



Corrective Regression Testing:

- S' == S & P' != P
- Same specification, new implementation.
- **Ex:** Refactoring or Non-functional Prop.

Progressive Regression Testing:

- S != S' (consequently, P' != P)
- New specification
- **Ex:** After adding new features.

© Francisco Oliveira Neto - 2022

How to do Regression Testing?

- Regression testing **should be automated**. Modifications occur often, so manual regression testing is infeasible.

- As software grows, regression test suites accumulate many tests.
- Running all tests (**retest all** approach) every time is **infeasible**.

- How can we make regression testing **cost-effective**?

© Francisco Oliveira Neto - 2018

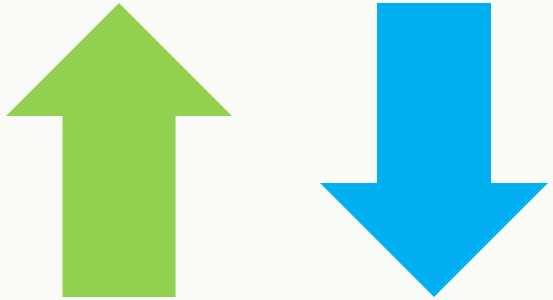
[1] Taming google-scale continuous testing. 2017. Memon AGao ZNguyen B et al.

How to make Regression Testing Cost-effective?

- More code churn = More testing needed
- **Not enough** resources (mostly **time**).
- Don't test less, **test smarter**!

… as large as **4.2 million tests** selected using reverse dependencies on changed source files since the previous milestone … In our work, we have observed unacceptably large delays of up to **9 hours.** [1]

Maximize Effectiveness



Minimize Resources

© Francisco Oliveira Neto - 2018

Test optimisation techniques

- Test **selection**

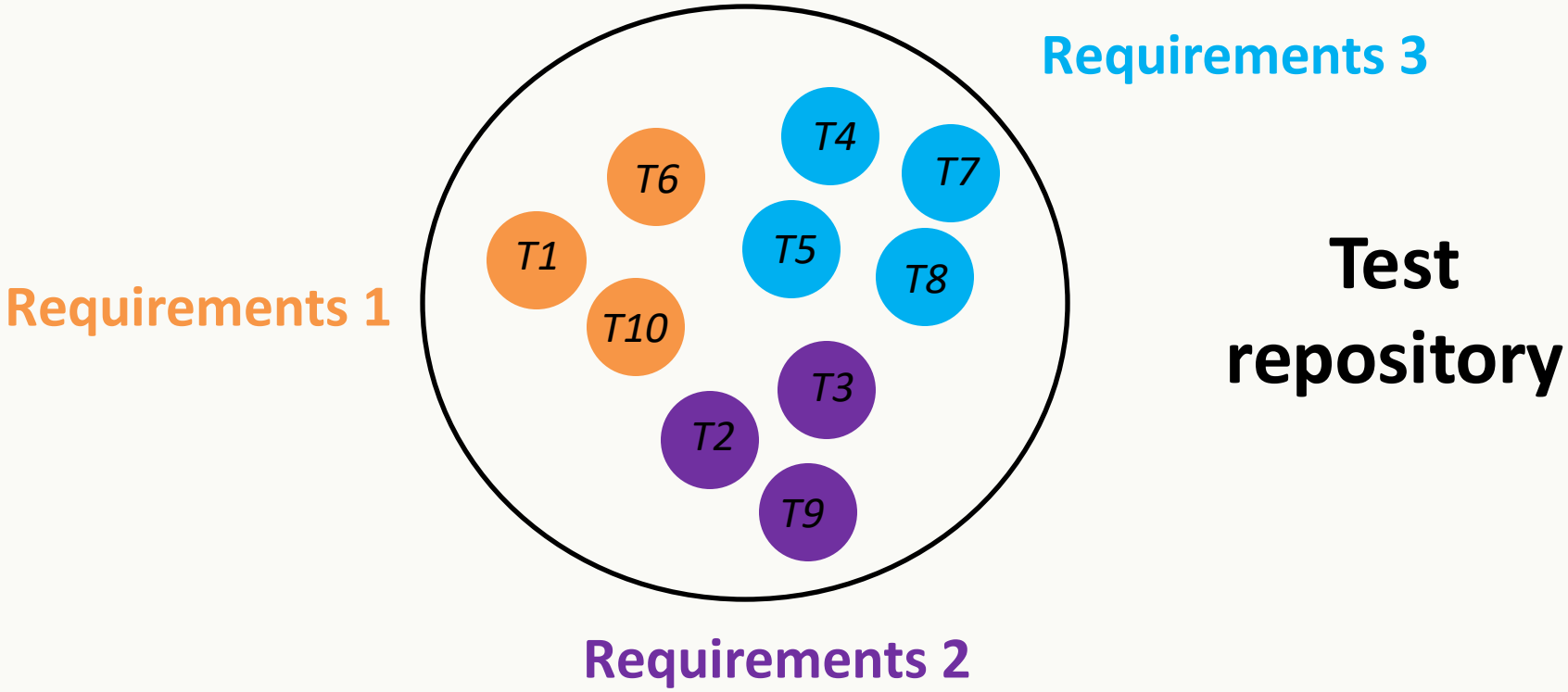
- Test **minimization**

- Test **prioritization**

© Francisco Oliveira Neto - 2019

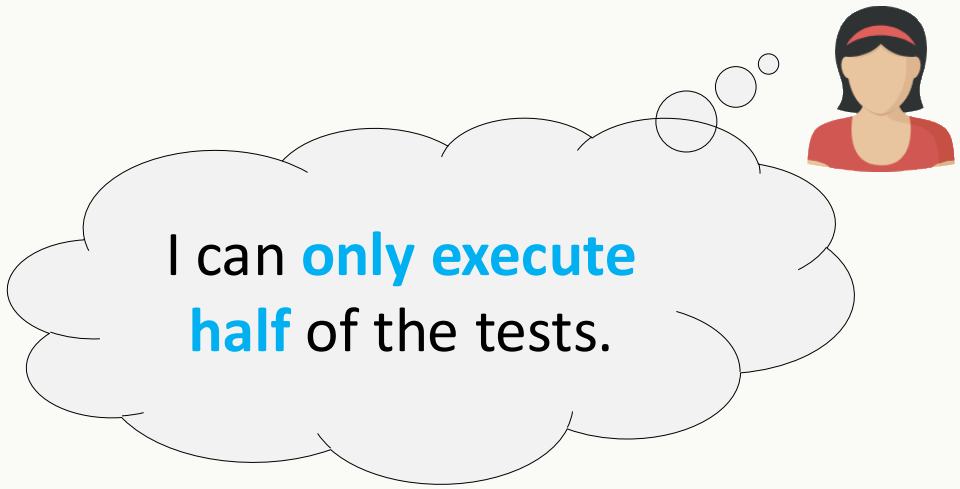
Test optimisation techniques

- Example:



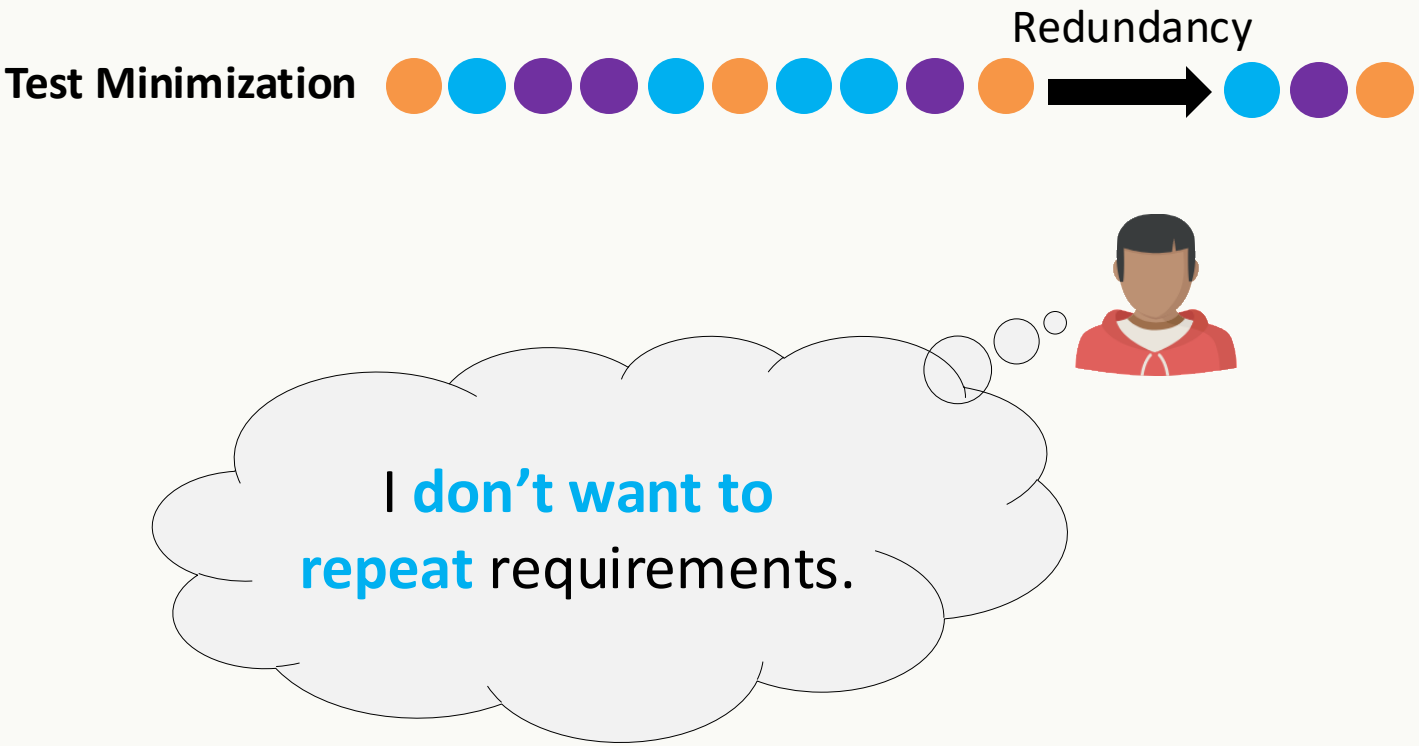
© Francisco Oliveira Neto - 2019

Example – Differences



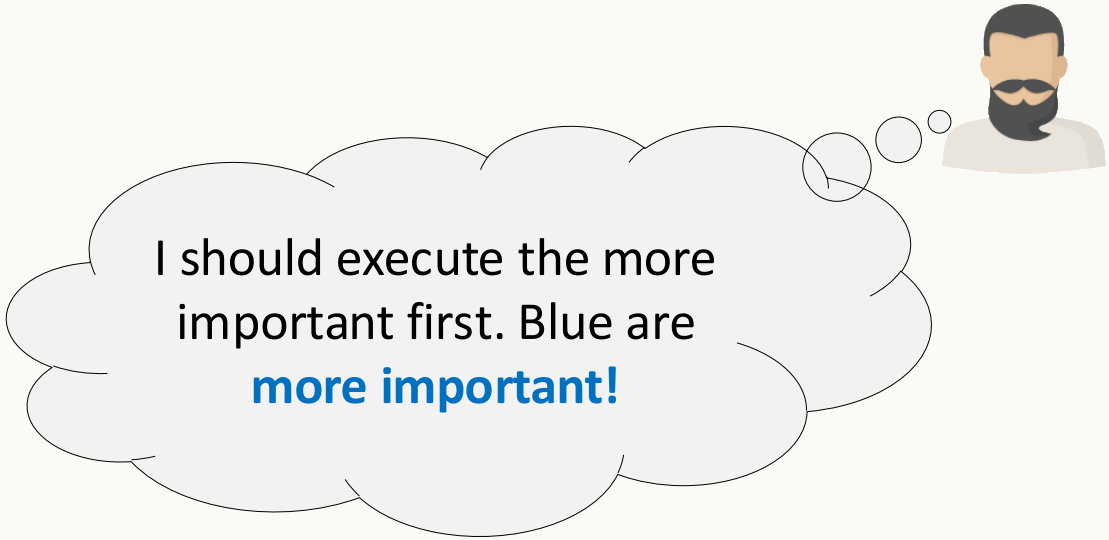
© Francisco Oliveira Neto - 2019

Example – Differences



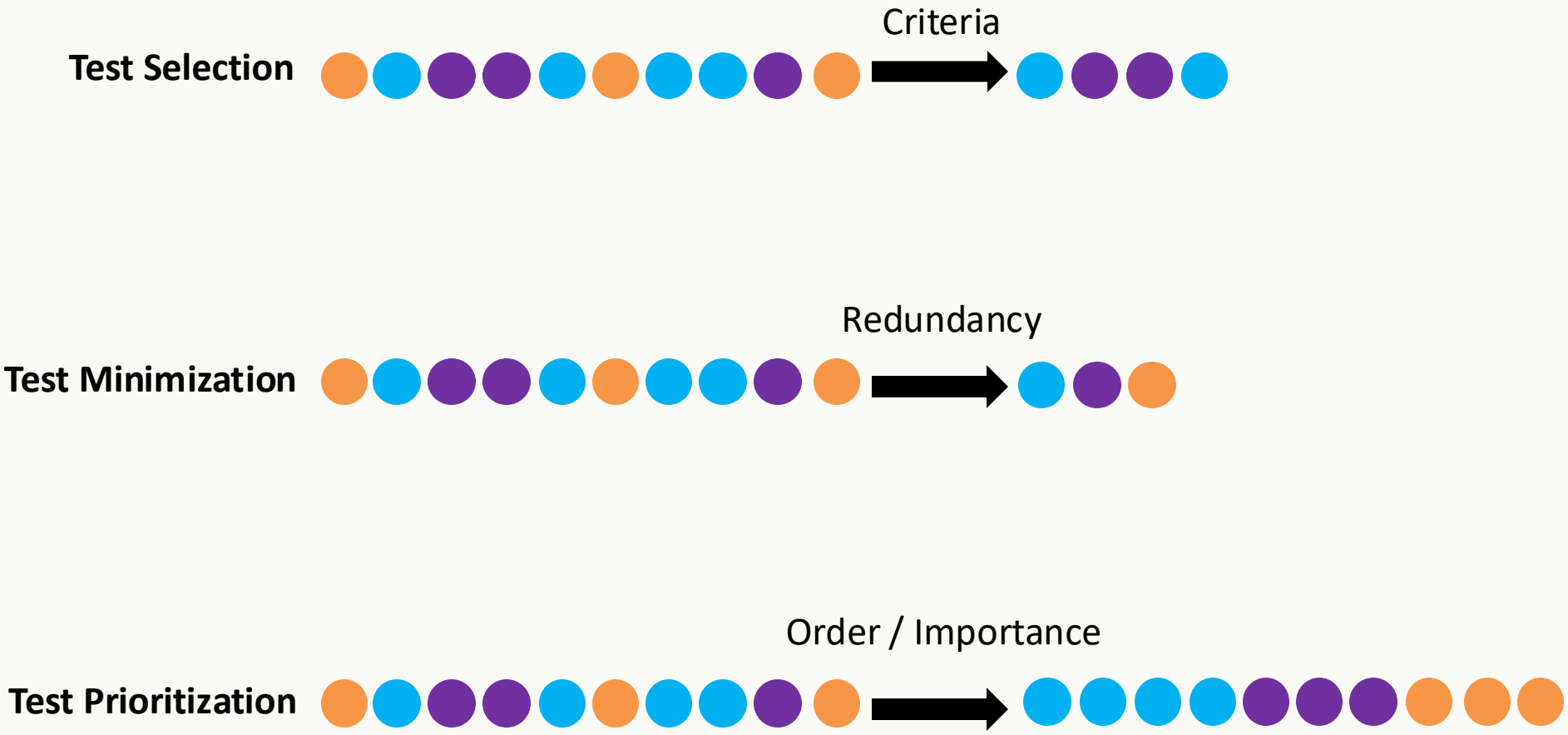
© Francisco Oliveira Neto - 2019

Example – Differences



© Francisco Oliveira Neto - 2019

Summary – Test Optimization

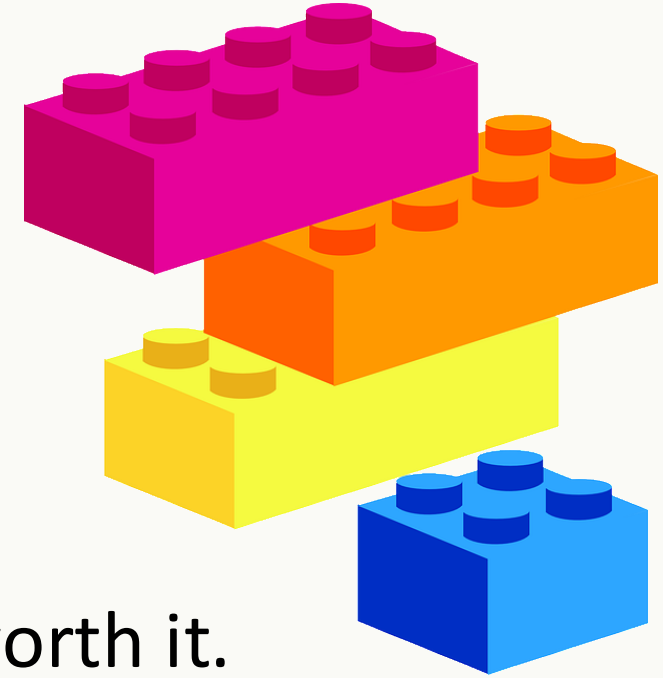


© Francisco Oliveira Neto - 2019

Img: https://goo.gl/Bkm1gW

When to use each?

- Selection: **Versatile** planning
- Minimisation: Remove/**Skip wasteful** info
- Prioritisation: Flexible **execution** costs
- In fact, its **good to combine them**
- **Important:** Any optimisation **must be cheaper** than the retest-all technique. Otherwise, not worth it.



© Francisco Oliveira Neto - 2019

Which criterias should we use to optimise?

- There are various **optimisation criteria** to choose:
  - Greedy coverage, Diversity-based, History-based, Commit-based, Time.
- We can optimise for several criteria with **multi-objective optimisation**.
  - Example: Test priority: (0.4 * time_optimisation) + (0.6 * code_coverage)
- The choice for criteria is dependent on the context and the information is **available** (modifications, execution time, etc.).

© Francisco Oliveira Neto - 2018

Cost-effective regression testing

- Benefits:
  - Test optimisation brings **awareness** on what needs to be tested.
  - Can **alleviate costs** with maintenance (identifying redundant tests).
  - Strategically plan for, e.g., ongoing **regression testing sessions**.

- Challenges:
  - The **discard risks**. Tests that were skipped might reveal failures.
  - Most of the techniques **must be automated**, otherwise its not worth it.

© Francisco Oliveira Neto - 2018

Part 1: Main take-away

- We use regression testing (RT) to identify **faults caused by modifications**.
- Ideally, we should re-test all. But its infeasible so we use test **optimisation to be cost-effective**.
- There are three ways to optimise regression testing: **selection, minimisation and prioritisation**. They can be combined

© Francisco Oliveira Neto - 2022

Exercise 1: Optimise the Test Suite

- Write a **minimised** test suite of statement coverage.
- A **prioritised** test suite according to two criteria:
  - Total coverage
  - Unique code coverage.

```
   procedure P
1. s1
2. while (c1) do
3.   if (c2) then
4.     exit
     else
5.     s3
     endif
6.   s4
   endwhile
7. s5
8. s6
9. s7
```

STATEMENT COVERAGE

statement	test case 1	test case 2	test case 3
1	X	X	X
2	X	X	X
3		X	X
4		X	
5			X
6			X
7	X		X
8	X		X
9	X		X

© Francisco Oliveira Neto - 2022

Exercise 1: Optimised Test Suite

- **Minimised Test Suite: [TC3, TC2]**. It covers all lines of code. TC1 covers the same lines as TC3, hence TC1 is redundant to TC3.

- **Total Coverage: [TC3, TC1, TC2]**. Total coverage is a “blind” and greedy technique. It only considers the total number of lines covered, even if those lines are redundant. **This is an offline prioritisation.**

- **Additional Coverage: [TC3, TC2, TC1].** For additional coverage, we order tests that will add to the current coverage. So, the choice of the next test depends on the previous. **This is an online prioritisation.**

STATEMENT COVERAGE

statement	test case 1	test case 2	test case 3
1	X	X	X
2	X	X	X
3		X	X
4		X	
5			X
6			X
7	X		X
8	X		X
9	X		X

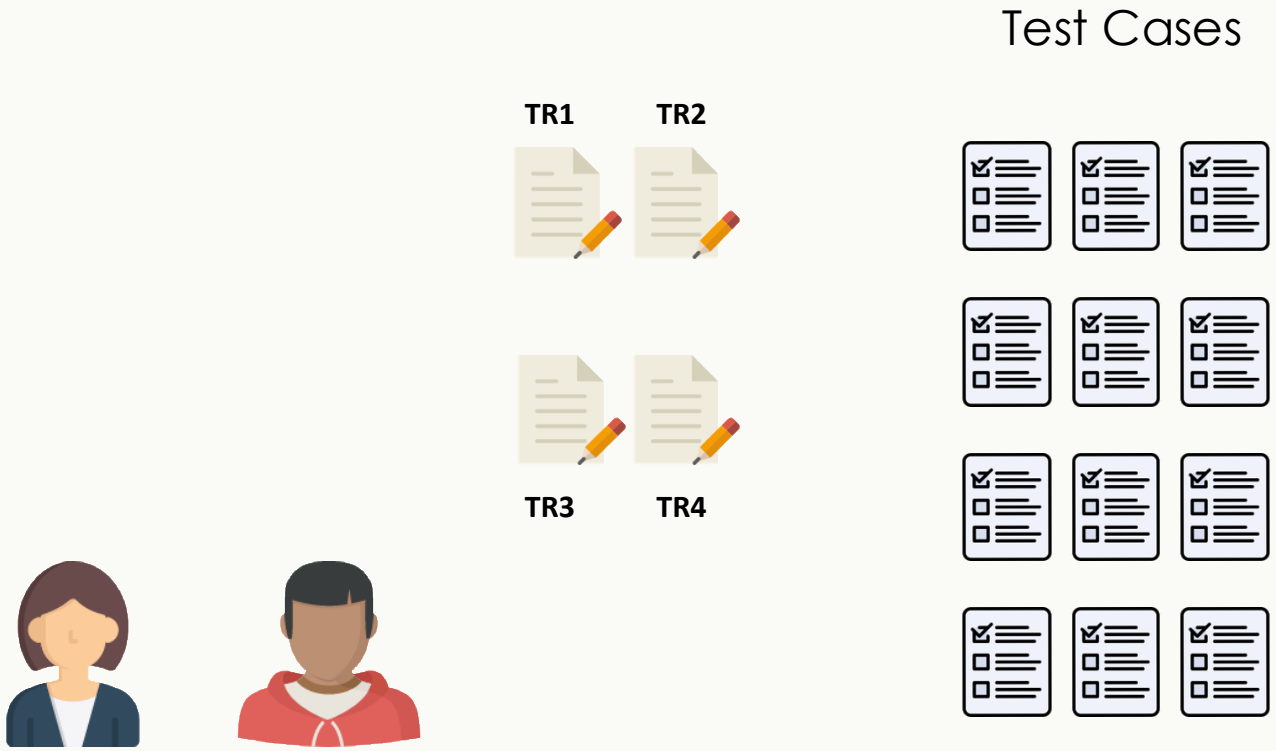
© Francisco Oliveira Neto - 2022

Part 2:

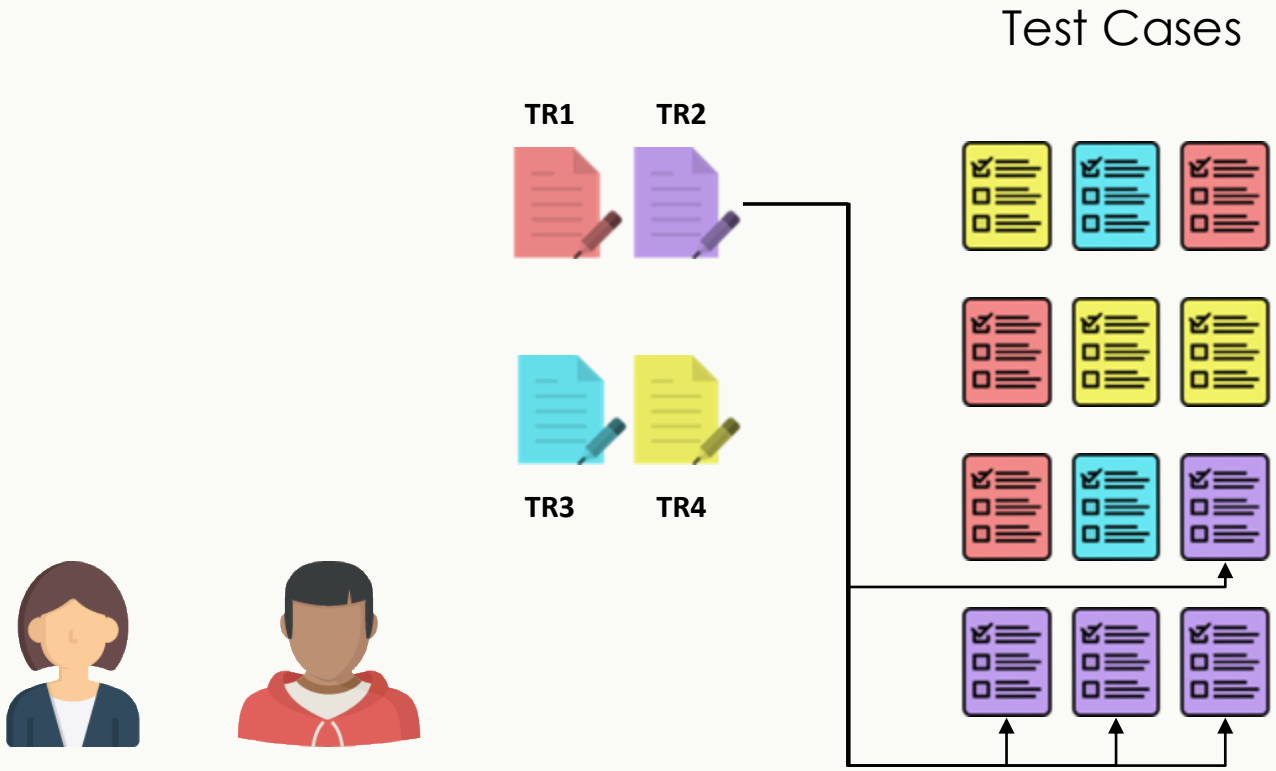
Test Suite Minimization Techniques

© Francisco Oliveira Neto - 2022

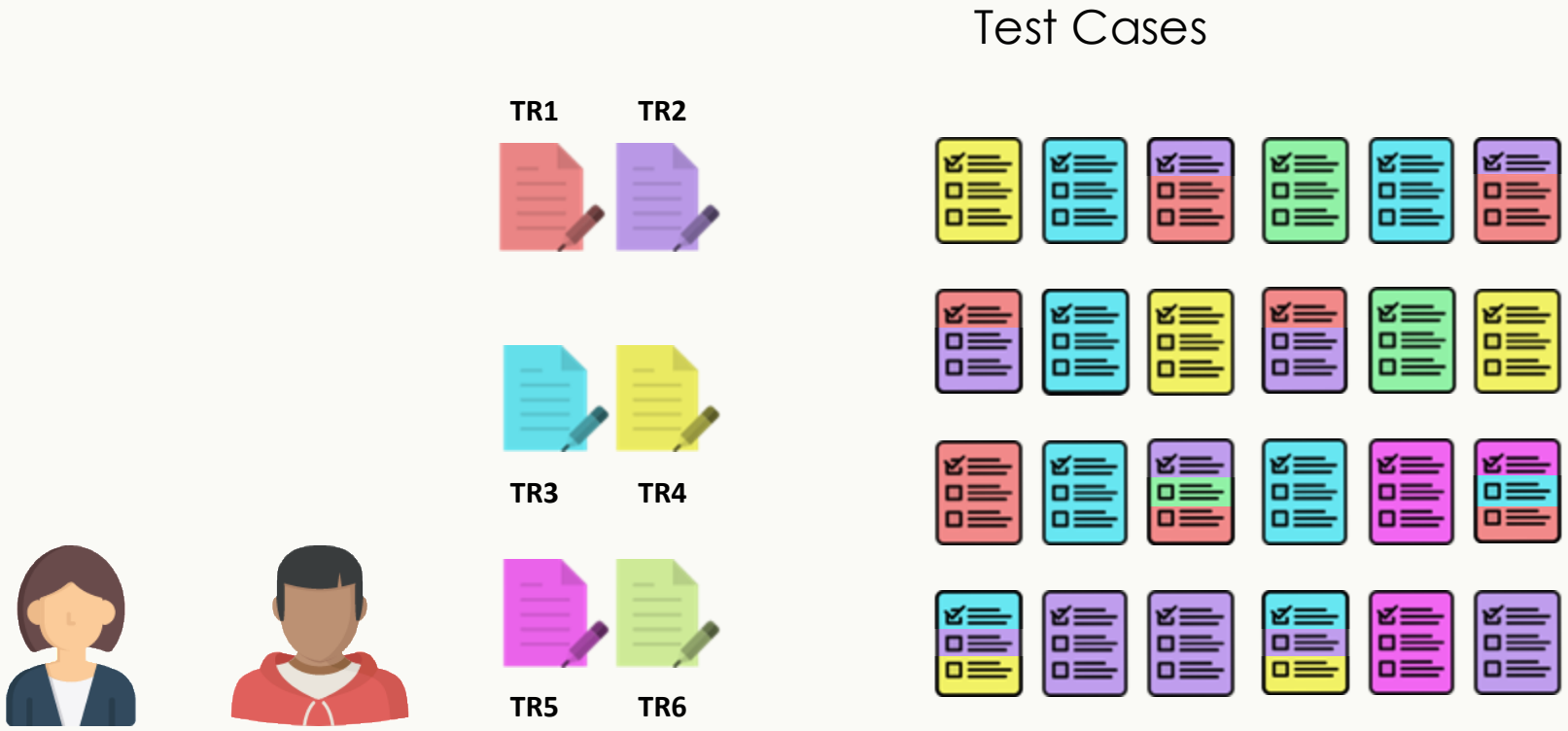
Test case redundancy



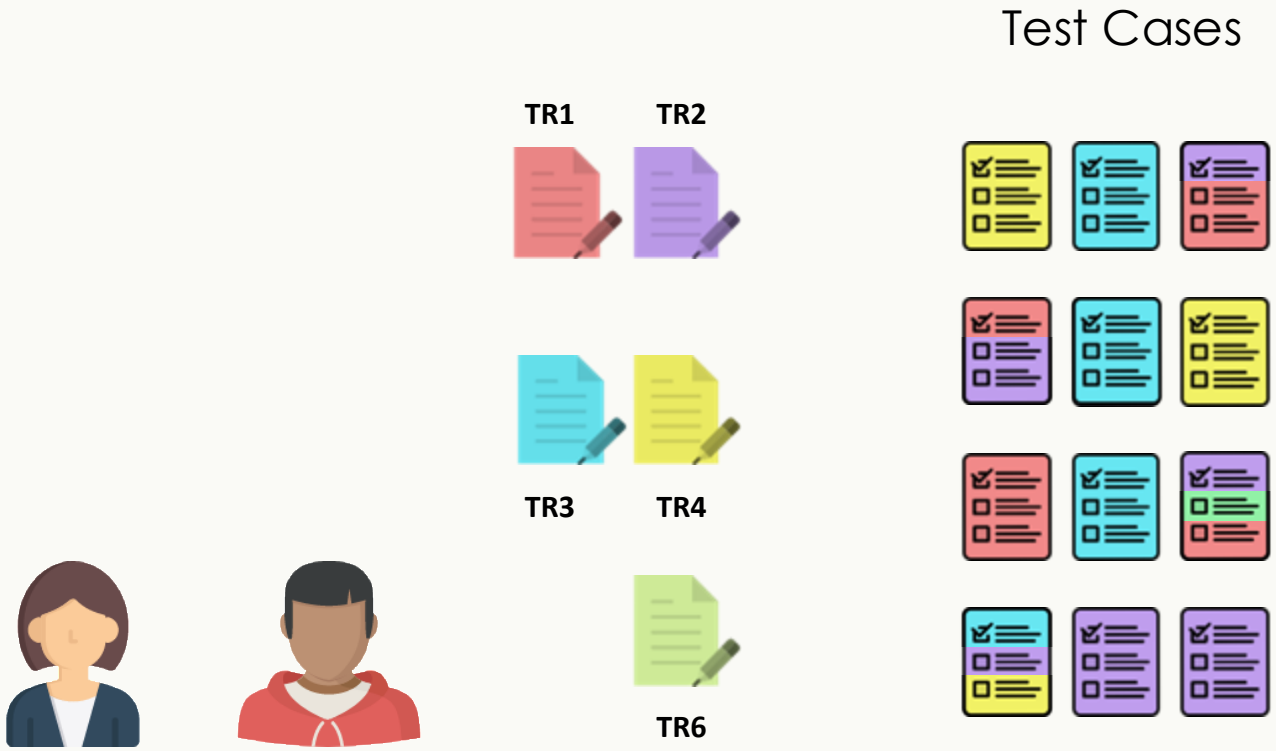
Test case redundancy



Test case redundancy



Essential test cases



Minimising Test Suites

		X	X			X		X	X		
		X	X					X	X	X	X
	X			X	X		X		X		
X					X						
								X			

Minimising Test Suites - G

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12
TR1			X	X			X		X	X		
TR2			X	X					X	X	X	X
TR3		X			X	X		X		X		
TR4	X					X						
TR5									X			
	1	1	2	2	1	2	1	1	3	3	1	1

T′ = { }

Minimising Test Suites - G

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12
TR1												
TR2												
TR3												
TR4	X					X						
TR5									X			
	1	0	0	0	0	1	0	0	1	0	0	0

T′ = { TC10 }

Minimising Test Suites - G

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12
TR1												
TR2												
TR3												
TR4	X					X						
TR5												
	1	0	0	0	0	1	0	0	0	0	0	0

T′ = { TC9, TC10 }

Minimising Test Suites - G

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12
TR1												
TR2												
TR3												
TR4												
TR5												
	0	0	0	0	0	0	0	0	0	0	0	0

T′ = { TC1, TC9, TC10 }

Minimising Test Suites

`T′ = { TC1, TC9, TC10 }` :Greedy minimisation

Minimising Test Suites - GE

Greedy

Essential

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12
TR1			X	X			X		X	X		
TR2			X	X					X	X	X	X
TR3		X			X	X		X		X		
TR4	X					X						
TR5									X			
	1	1	2	2	1	2	1	1	3	3	1	1

T′ = { }

Minimising Test Suites - GE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1			X	X			X		X	X			5
TR2			X	X					X	X	X	X	6
TR3		X			X	X		X		X			5
TR4	X					X							2
TR5									X				1
	1	1	2	2	1	2	1	1	3	3	1	1	

T′ = { }

Minimising Test Suites - GE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1													0
TR2													0
TR3		X			X	X		X		X			5
TR4	X					X							2
TR5													0
	1	1	0	0	1	2	0	1	0	1	0	0	

T′ = {TC9 }

Minimising Test Suites - GE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1													0
TR2													0
TR3		X			X	X		X		X			0
TR4	X					X							0
TR5													0
	0	0	0	0	0	0	0	0	0	0	0	0	

T′ = { TC6, TC9}

Minimising Test Suites

T′ = { TC1, TC9, TC10 } :Greedy minimisation

T′ = {TC6, TC9 } :Greedy-Essential minimisation

Minimising Test Suites - GRE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1			X	X			X		X	X			5
TR2			X	X					X	X	X	X	6
TR3		X			X	X		X		X			5
TR4	X					X							2
TR5									X				1
	1	1	2	2	1	2	1	1	3	3	1	1	

Greedy

1-to-1 **R**edundant

Essential

T′ = { }

Minimising Test Suites - GRE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1			X	X			X		X	X			5
TR2			X	X					X	X	X	X	6
TR3		X			X	X		X		X			5
TR4	X					X							2
TR5									X				1
	1	1	2	2	1	2	1	1	3	3	1	1	

$T' = \{ \quad \}$

$TC1 \subseteq TC6$
$TC2 \subseteq TC6$
$TC5 \subseteq TC6$
$TC8 \subseteq TC6$

$TC3 \subseteq TC9$
$TC4 \subseteq TC9$
$TC7 \subseteq TC9$

$TC3 \subseteq TC10$
$TC4 \subseteq TC10$
$TC11 \subseteq TC10$
$TC12 \subseteq TC10$

Minimising Test Suites - GRE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1									X	X			**2**
TR2									X	X			**2**
TR3						X				X			**2**
TR4						X							**1**
TR5									X				**1**
						2			**3**	**3**			

T′ = { }

Minimising Test Suites - GRE

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10	TC11	TC12	
TR1													0
TR2													0
TR3													0
TR4													0
TR5													0
						0			0	0			

T′ = { TC6, TC9 }

Minimising Test Suites

T′ = { TC1, TC9, TC10 } :Greedy (G)

T′ = {TC6, TC9 } :Greedy-Essential (GE)

T′ = {TC6, TC9 } :Greedy-Redundant-Essential (GRE)

Test Suite Minimisation

Advantages:

- Safe coverage of test requirements
- **Meaningful feedback** on coverage
- Provide **insights** on test case design

Disadvantages:

- **May not** provide significant size reduction
- Requires **traceability support**

Exercise – Practicing Minimization

- Find the minimized test suite using: G, GE, GRE.

Req. \ TC	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10
R1	✓			✓		✓				
R2		✓		✓	✓					
R3			✓		✓		✓			
R4	✓			✓				✓		
R5		✓							✓	
R6										✓
R7	✓		✓							

© Francisco Oliveira Neto - 2022